

Manual Técnico

Geovisor Ecopedagógico de Sevilla, Valle del Cauca

Cristóbal Valencia Cerón y José David Molina Delgado

Facultad de Ingeniería, Universidad Santiago de Cali

Programa de Ingeniería de Sistemas

Trabajo de grado

Director: Diego Fernando Loaiza — Grupo de Investigación INFORMA

2026

Resumen

Este documento describe la arquitectura, los componentes, el modelo de datos y el flujo de despliegue del Geovisor Ecopedagógico de Sevilla, Valle del Cauca. La aplicación es una single page application que se ejecuta íntegramente en el navegador: no dispone de servidor propio, interfaz de programación de aplicaciones ni base de datos. Consume geoservicios públicos conformes al estándar Web Map Service y archivos GeoJSON estáticos versionados en el repositorio. La exposición avanza de lo general a lo particular, apoyada en doce figuras y siete tablas, e incluye un anexo con los veintiún requisitos funcionales y sus criterios de aceptación verificables.

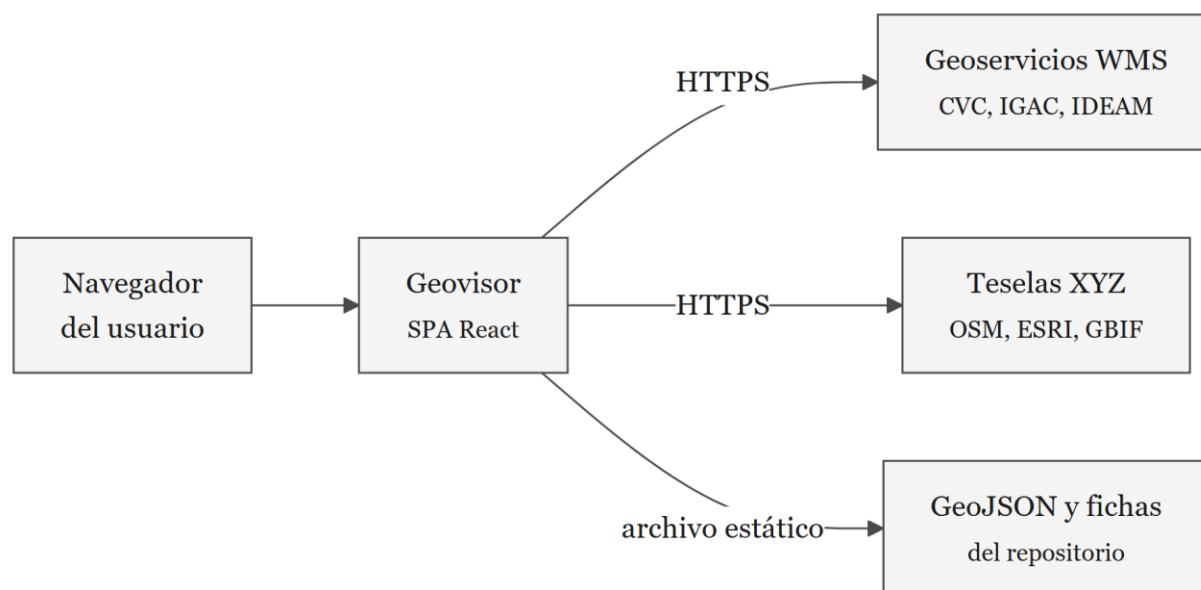
Palabras clave: arquitectura de software, sistemas de información geográfica, aplicación web progresiva, WMS, JAMstack

Visión General de la Arquitectura

El Geovisor es una single page application que se ejecuta al cien por ciento en el cliente. Todo ocurre en el navegador, que consume geoservicios públicos y archivos estáticos versionados junto al código. Esta decisión elimina el costo de operación de un servidor y el riesgo asociado a custodiar datos personales, a cambio de depender de la disponibilidad de terceros para las capas ráster.

Figura 1

Vista de arquitectura de alto nivel

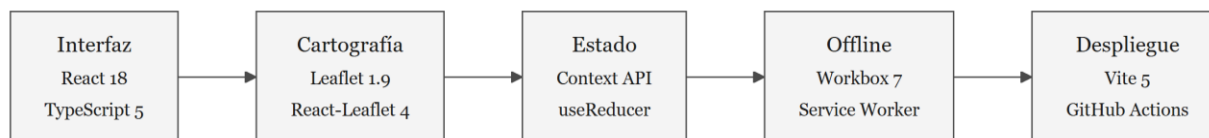


Nota. El navegador es el único entorno de ejecución; los geoservicios y las telas son dependencias externas.

Stack Tecnológico

Figura 2

Capas del stack tecnológico



Nota. Cada capa depende únicamente de la anterior, lo que permite sustituirlas de forma aislada.

Tabla 1

Componentes del stack y su versión

Capa	Tecnología	Versión
Interfaz	React + TypeScript	18.x / 5.x
Empaquetado	Vite	5.x
Motor cartográfico	Leaflet + React-Leaflet	1.9 / 4.x
Estilos	Tailwind CSS	3.x
Primitivos accesibles	Radix UI	1.x
Ruteo	React Router	6.x
Estado	Context API + useReducer	—
Offline	vite-plugin-pwa + Workbox	7.x
Geometría	Turf.js	7.x
Exportación de imagen	html2canvas	1.4
Calidad	ESLint + Prettier + Husky	—
Despliegue	GitHub Actions	—

Nota. El estado no se persiste en almacenamiento local: se serializa en los parámetros de la dirección web, lo que permite compartir una vista.

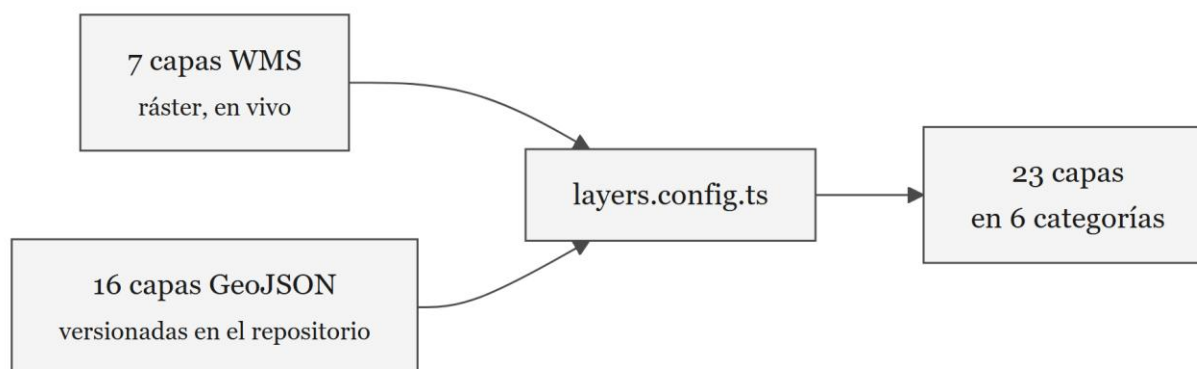
Origen y Estrategia de Datos

Las capas se consumen de dos formas según su naturaleza. Los vectores pequeños se sirven como GeoJSON estático versionado en el repositorio, lo que garantiza su disponibilidad sin conexión. Las capas ráster —coberturas, clima, conflictos de uso— se consultan en vivo

mediante Web Map Service, porque su tamaño y su frecuencia de actualización desaconsejan replicarlas.

Figura 3

Composición del inventario de capas por tipo de servicio



Nota. Las 23 capas se declaran en un único archivo de configuración, con independencia de su origen.

Tabla 2

Fuentes de datos e instituciones proveedoras

Institución	Tipo	Contenido
CVC — Portal GeoCVC	WMS	Cuencas, coberturas, conflictos de uso
IGAC	WMS	Cartografía base y catastral
IDEAM	WMS	Isoyetas y estaciones hidroclimatológicas
Instituto Humboldt	WMS	Delimitación de páramos
RUNAP	WMS	Áreas protegidas
GBIF / SiB Colombia	XYZ	Registros de presencia de especies
OpenStreetMap	XYZ	Mapa base callejero
ESRI World Imagery	XYZ	Mapa base satelital

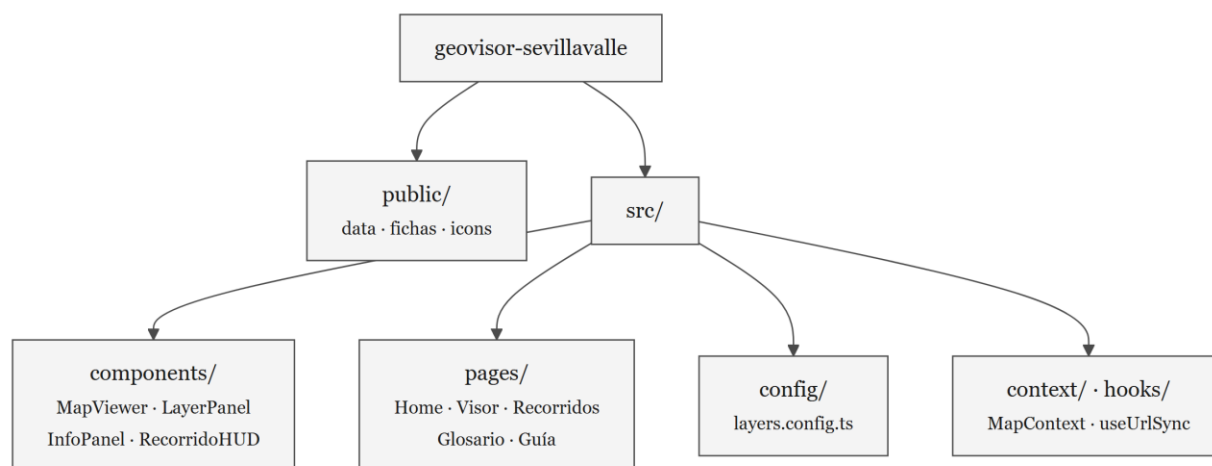
Nota. Todas las fuentes son de acceso público y conservan sus licencias de origen.

Tabla 3*Distribución de las capas por categoría*

Categoría	Capas	Tipo de dato
Actores sociales	4	GeoJSON
Agua	6	GeoJSON y WMS
Biodiversidad	6	GeoJSON y WMS
Cambio climático	3	GeoJSON y WMS
Suelos	1	WMS
Territorio	3	GeoJSON y WMS

Nota. Total: 23 capas, de las cuales 16 son GeoJSON estático y 7 son WMS.

Estructura del Proyecto

Figura 4*Organización de carpetas y módulos*

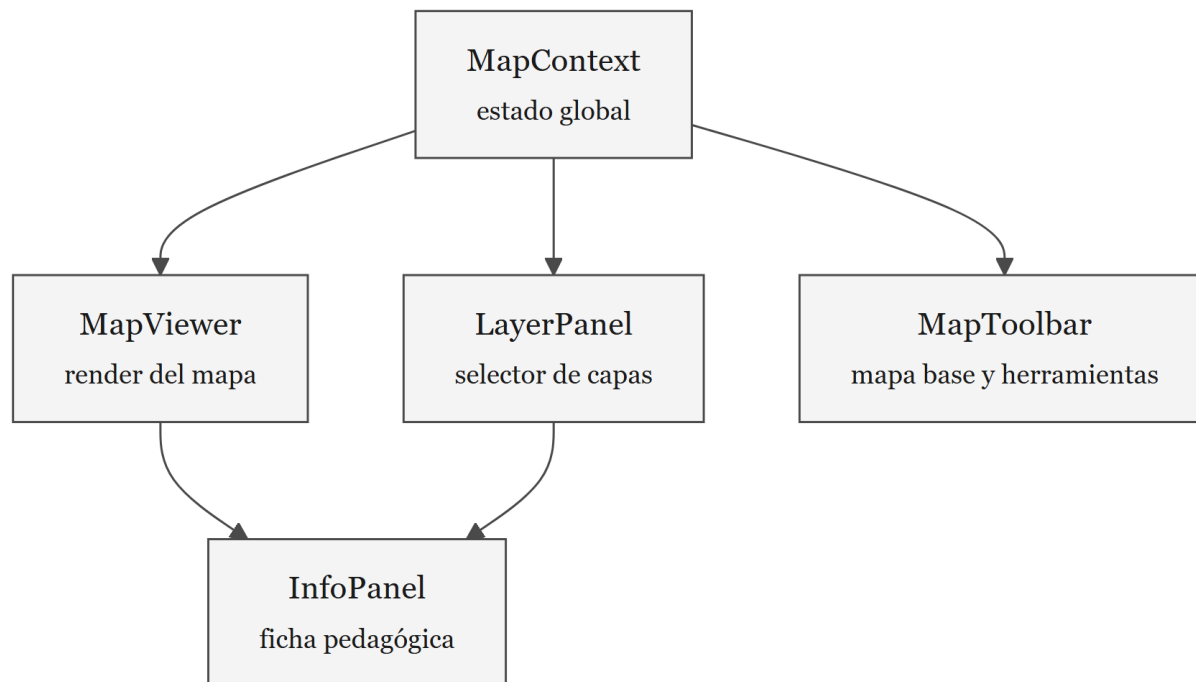
Nota. La configuración de capas está aislada en un único módulo, de modo que añadir una capa no requiere modificar ningún componente.

Componentes de la Interfaz

El estado global reside en `MapContext` y lo consumen tanto el panel de capas como el visor del mapa. Los componentes no se comunican entre sí directamente: lo hacen a través del estado compartido o, cuando eso no es posible, mediante eventos.

Figura 5

Componentes React y su relación con el estado global



Nota. InfoPanel se alimenta tanto del clic sobre el mapa como de la selección en el panel de capas.

Tabla 4

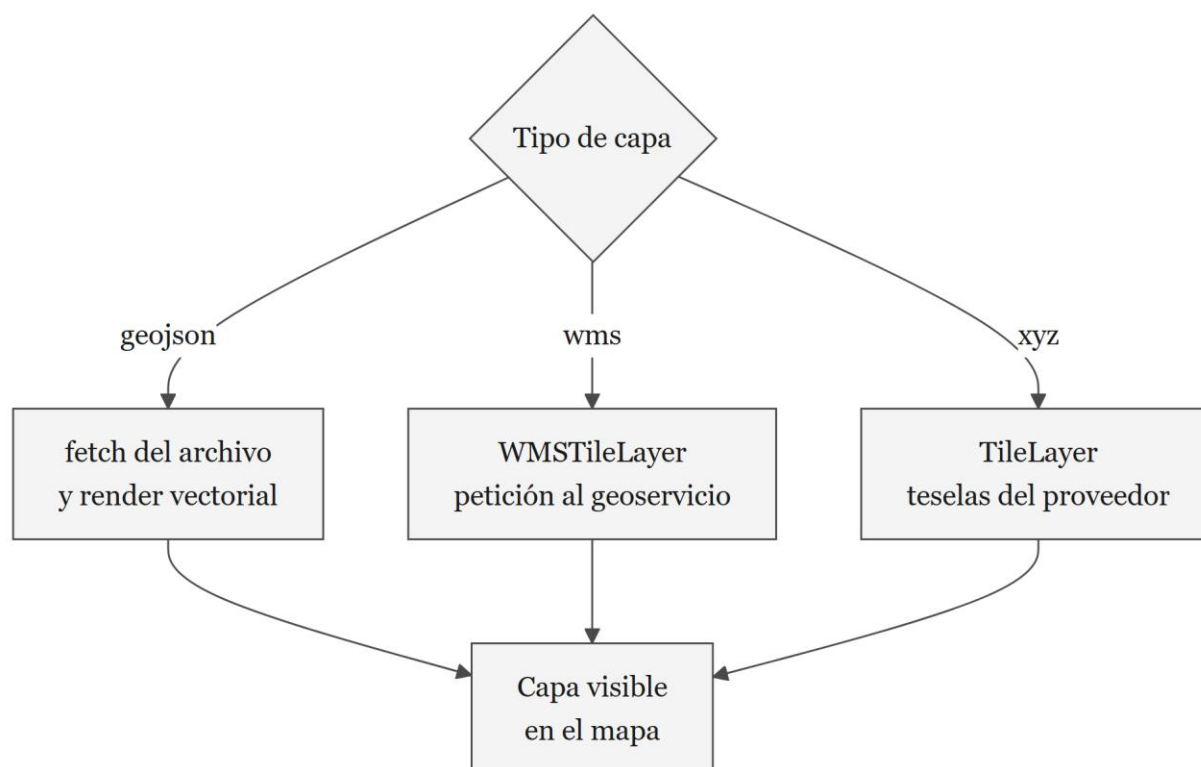
Responsabilidad de los componentes principales

Componente	Responsabilidad
MapViewer	Contenedor Leaflet; renderiza GeoJSON, WMS y teselas
LayerPanel	Acordeón de categorías, activación y opacidad de capas
InfoPanel	Panel lateral con la ficha pedagógica
MapToolbar	Mapa base, medición, exportación y búsqueda
RecorridoHUD	Navegación de las paradas de un recorrido
FeatureSearchPanel	Búsqueda por atributos en las capas activas

Flujo de Carga de una Capa

Figura 6

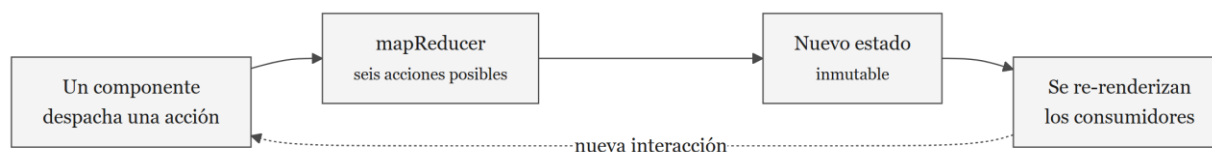
Ruta de carga según el tipo de capa



Nota. El tipo declarado en la configuración determina el componente de Leaflet que se instancia.

Gestión de Estado

El estado global se maneja con un reducer. Cada interacción despacha una acción, el reducer produce un estado nuevo e inmutable y los componentes suscritos se vuelven a renderizar. No se emplea ninguna biblioteca externa de gestión de estado.

Figura 7*Ciclo de actualización del estado global*

Nota. El flujo es unidireccional: ninguna vista modifica el estado directamente.

Tabla 5*Acciones del reducer de MapContext*

Acción	Efecto sobre el estado
TOGGLE_LAYER	Añade o quita una capa del listado de capas activas
SET_LAYER_OPACITY	Fija la opacidad de una capa concreta
SET_MAPA_BASE	Cambia el mapa base entre callejero, satélite y topográfico
SET_ZOOM	Actualiza el nivel de acercamiento
SET_CENTER	Actualiza el punto central del mapa
SET_STATE_FROM_URL	Restaura el estado a partir de la dirección web

Nota. El encuadre a la extensión de una capa y la apertura de la ficha no pasan por el reducer: se resuelven mediante eventos, como se detalla en la sección siguiente.

Comunicación entre Componentes

Los componentes que viven fuera del árbol del contenedor del mapa —como la barra de navegación o el panel de capas— no pueden acceder a la instancia de Leaflet mediante el hook correspondiente. La solución adoptada fue desacoplarlos mediante eventos personalizados de la ventana, que unos controladores situados dentro del mapa escuchan y traducen a llamadas de Leaflet.

Figura 8

Comunicación entre componentes mediante eventos de la ventana

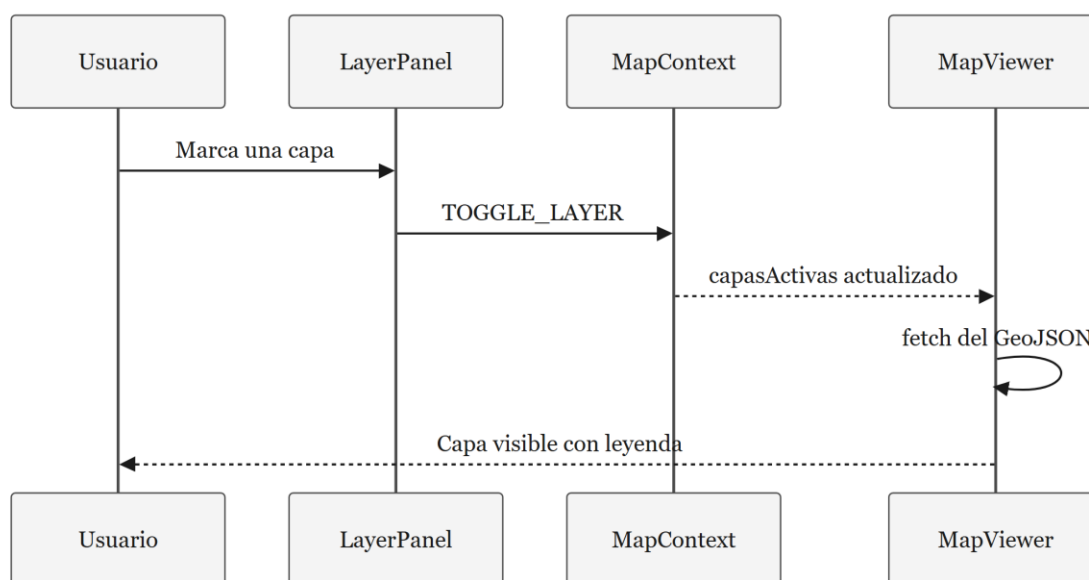


Nota. Este mecanismo se emplea para el encuadre a una capa y para la apertura de la ficha pedagógica.

Secuencia de Activación de una Capa

Figura 9

Secuencia completa de activación de una capa GeoJSON



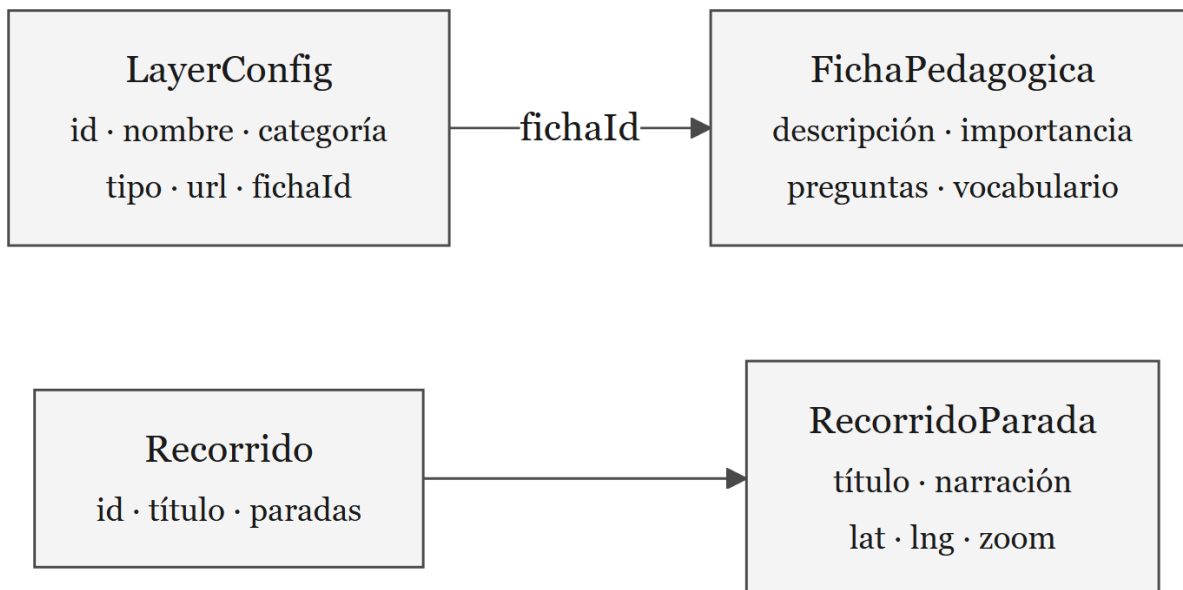
Nota. El diagrama recoge el recorrido extremo a extremo, desde el clic del usuario hasta la capa visible.

Modelo de Datos

El modelo se apoya en dos estructuras centrales: la configuración declarativa de las capas y la ficha pedagógica, un archivo por capa. Los recorridos guiados constituyen una tercera estructura independiente.

Figura 10

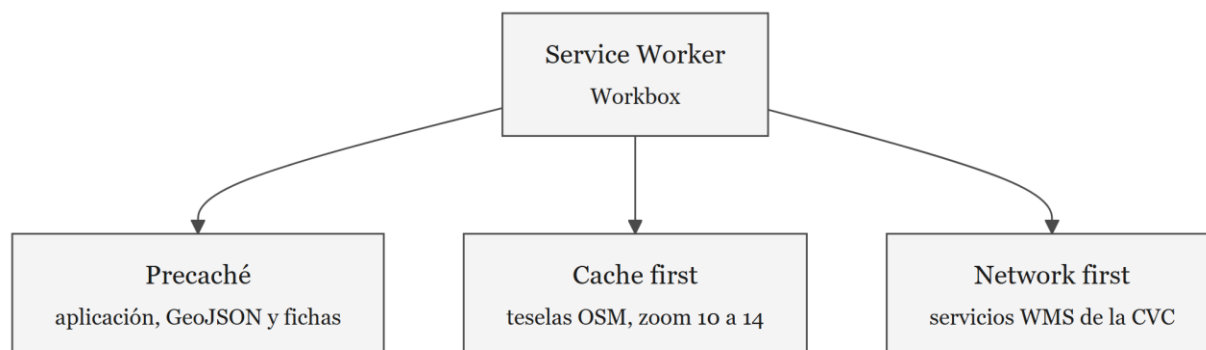
Relación entre configuración de capas, fichas y recorridos



Nota. El campo fichaId enlaza cada capa con su contenido pedagógico.

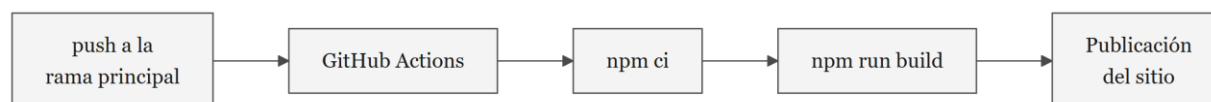
Capa sin Conexión

El Service Worker aplica una estrategia de caché diferenciada según el tipo de recurso. La aplicación, los archivos GeoJSON y las fichas se precargan en la instalación. Las teselas del mapa base se sirven primero desde caché. Los servicios WMS se consultan primero en red y solo recurren a la caché si esta no responde.

Figura 11*Estrategia de caché del Service Worker*

Nota. Las teselas precargadas cubren el área del municipio entre los niveles de acercamiento 10 y 14, aproximadamente 50 MB.

Despliegue e Integración Continua

Figura 12*Flujo de integración y despliegue continuo*

Nota. El proceso es automático: no requiere intervención manual tras la publicación de cambios.

Requisitos No Funcionales

Tabla 6

Requisitos no funcionales y su estado

Atributo	Objetivo	Estado
Rendimiento	Bundle JavaScript menor a 500 KB comprimido	142 KB
Rendimiento	Primer contenido visible en menos de 1,5 s	Por auditar
Datos	Cada archivo GeoJSON menor a 2 MB	Cumplido
Offline	Teselas del municipio, niveles 10 a 14	Cumplido
Accesibilidad	WCAG 2.1 nivel AA, contraste mínimo 4.5:1	En progreso
Seguridad	Solo HTTPS y sin datos personales	Cumplido
Mantenibilidad	Capas declaradas en un único archivo	Cumplido

Nota. La cifra de rendimiento corresponde a la medición del build de producción vigente en la fecha de este documento.

Referencias

Agafonkin, V. (2024). Leaflet: An open-source JavaScript library for mobile-friendly interactive maps (Versión 1.9) [Software]. <https://leafletjs.com>

Corporación Autónoma Regional del Valle del Cauca. (2024). Portal GeoCVC.
<https://geo.cvc.gov.co/inicio/>

Open Geospatial Consortium. (2010). OpenGIS Web Map Service (WMS) implementation specification (Versión 1.3.0). <https://www.ogc.org/standards/wms>

Vite Team. (2024). Vite: Next generation frontend tooling (Versión 5) [Software].
<https://vitejs.dev>

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1.
<https://www.w3.org/TR/WCAG21/>

Anexo A: Requisitos Funcionales y Criterios de Aceptación

Se listan los veintiún requisitos funcionales agrupados por módulo, con su prioridad y su criterio de aceptación verificable, según el Documento de Requerimientos v2.0.

Módulo de visualización

Tabla 7

Requisitos funcionales del módulo de visualización

ID	Nombre	Prior.	Criterio de aceptación
RF-01	Mapa base interactivo	Alta	Mapa cargado en menos de 3 s; respuesta al zoom en menos de 100 ms
RF-02	Panel de capas jerárquico	Alta	Al alternar una capa, aparece o desaparece en menos de 500 ms
RF-03	Control de opacidad	Media	Ajuste fluido, sin retardo perceptible
RF-04	Leyenda dinámica	Alta	Leyenda visible con la capa; la imagen WMS carga en menos de 2 s
RF-05	Zoom a extensión de capa	Media	Desplazamiento animado de 300 ms
RF-06	Selector de mapa base	Alta	Cambio en menos de 1 s sin reiniciar las capas activas

Módulo de consulta

Tabla 8

Requisitos funcionales del módulo de consulta

ID	Nombre	Prior.	Criterio de aceptación
RF-07	Consulta de atributos al clic	Alta	Globo de información en menos de 200 ms para capas GeoJSON
RF-08	Búsqueda de lugares	Media	Sugerencias en menos de 1 s a partir de 3 caracteres
RF-09	Búsqueda en capas activas	Media	Resultados en tiempo real; resalta el elemento en el mapa
RF-10	Herramienta de medición	Media	Cálculo con Turf.js; error máximo del 0,01 % frente a referencia
RF-11	Coordenadas del cursor	Baja	Actualización continua con precisión de cuatro decimales

Módulo pedagógico

Tabla 9

Requisitos funcionales del módulo pedagógico

ID	Nombre	Prior.	Criterio de aceptación
RF-12	Fichas pedagógicas	Alta	Al menos 150 palabras, una imagen y tres preguntas por ficha
RF-13	Contenido multimedia	Alta	Imágenes con carga diferida; videos sin reproducción automática
RF-14	Modo recorrido guiado	Media	Avance con Siguiente y Anterior; desplazamiento de 800 ms
RF-15	Línea de tiempo	Media	Pendiente: no hay series históricas disponibles
RF-16	Glosario territorial	Baja	Buscable por texto; enlaza con la capa relacionada

Módulos de exportación, accesibilidad y uso sin conexión

Tabla 10

Requisitos funcionales de exportación, accesibilidad y uso sin conexión

ID	Nombre	Prior.	Criterio de aceptación
RF-17	Exportar vista como imagen	Media	Descarga en menos de 3 s con resolución mínima de 1920×1080 px
RF-18	Compartir dirección de la vista	Media	Abrir la dirección en otro navegador restaura el mismo estado
RF-19	Modo sin conexión	Alta	Tras la primera visita se explora sin red; avisa si la WMS no responde
RF-20	Texto alternativo de imágenes	Alta	Auditoría automatizada sin violaciones de tipo image-alt
RF-21	Navegación por teclado	Alta	Todos los controles operables con teclado y foco visible